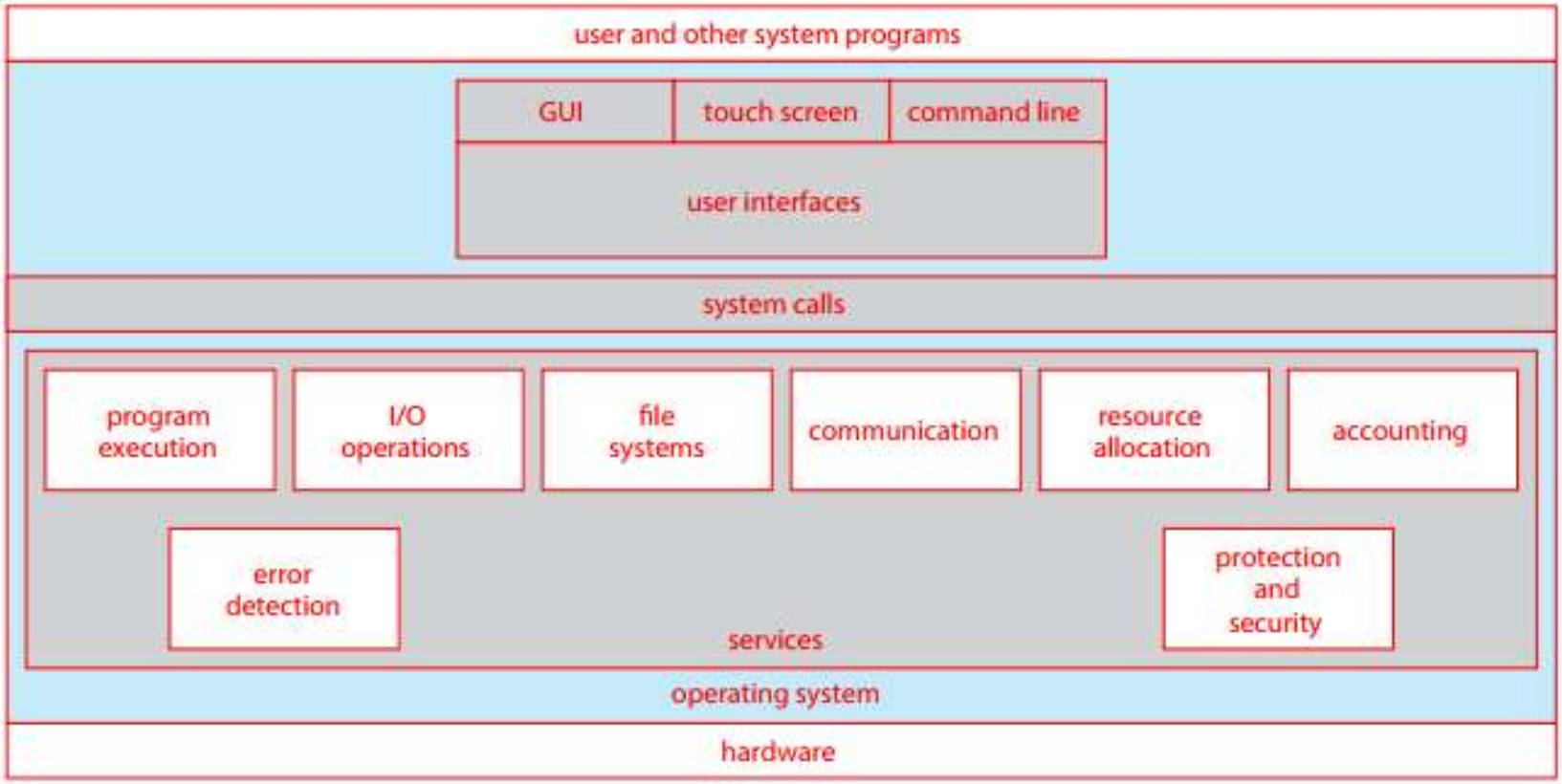
Operating System Structure

Introduction

- An operating system is made up of various structures and components
- Services provided by the OS, the interfaces it presents to users and applications, and the design principles that guide its implementation
- Also look at how the OS is built, booted, and debugged

Operating System Services



Operating-System Services – User Services

- **User Interface (UI)**
  - Provides a way for users to interact with the system
  - This could be in the form of CLI, GUI or Touch-Screen
- **Program Execution**
  - Loads a program into memory, runs it, and handles its termination (normal or abnormal)
- **I/O Operations**
  - Manages I/O to files and devices
  - Users cannot control devices directly for protection
- **File-System Manipulation**
  - Programs can create, delete, read, write, and manipulate files and directories
  - File permissions management to allow or deny access to files or directories based on file ownership
- **Communications**
  - Enables processes to exchange information
  - Normally via shared memory or message passing technique
- **Error Detection**
  - The OS constantly checks for and handles errors in hardware, I/O devices, and user programs

Operating System Services – Services for System Efficiency

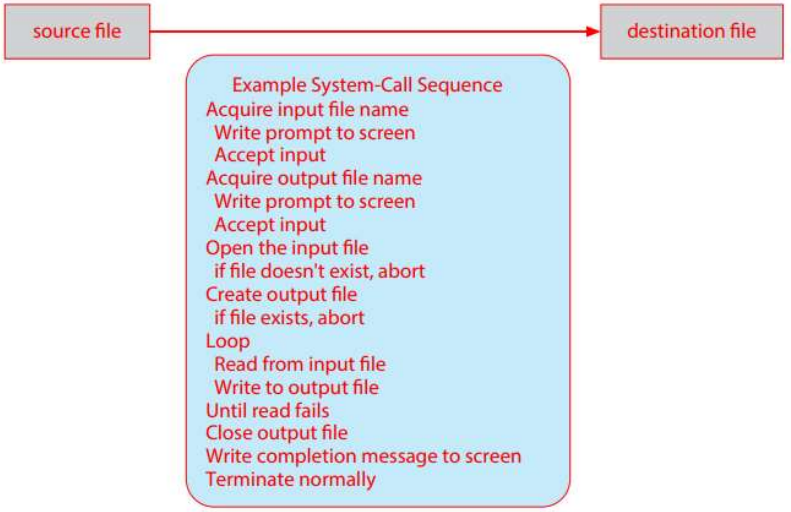
- **Resource Allocation**
  - Allocates and manages resources like CPU cycles, memory, and devices among multiple users and processes
- **Accounting**
  - Tracks resource usage for billing or performance monitoring
- **Protection and Security**
  - Protects system resources from unauthorized access and defends against external attacks

User and Operating-System Interface

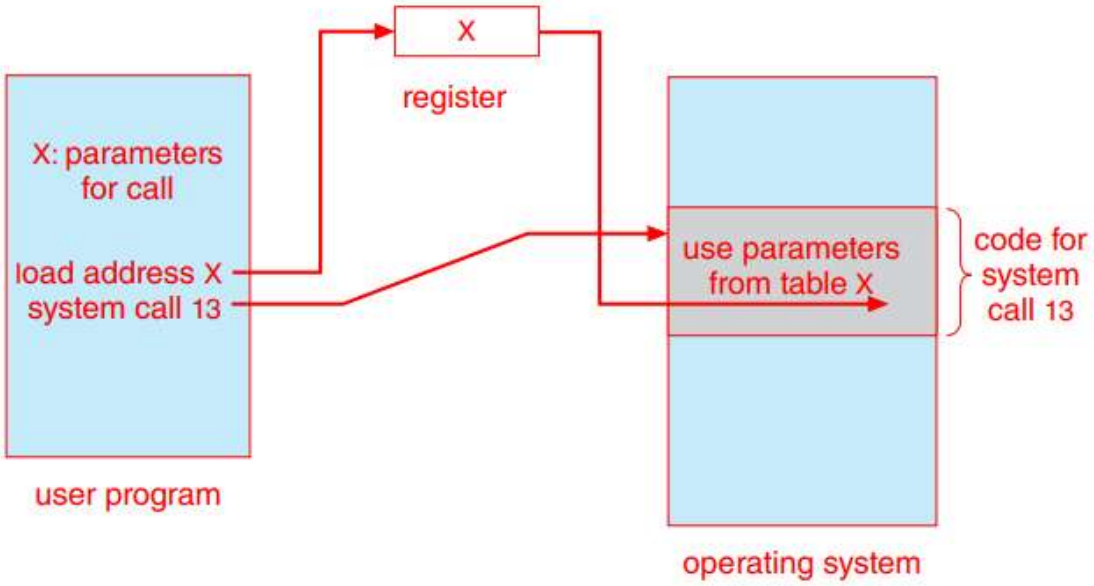
- **Command-Line Interface (CLI):**
  - The command interpreter as a special program that is running when a process is initiated or when a user first logs on in an interactive system
  - Accepts text commands from the user (e.g., shells like bash in Linux)
  - The command interpreter can either contain the code for the commands or use the command name to load and execute a program
  - Efficient for repetitive tasks and scripting
- **Graphical User Interface (GUI):**
  - A mouse-based window-and-menu system with a desktop metaphor
  - Users interact with icons representing programs and files
- **Touch-Screen Interface:**
  - Used on mobile devices like smartphones and tablets
  - Users interact with gestures like pressing and swiping

System Calls

- The primary interface to the services made available by the operating system
  - Calls are generally available as functions written in C and C++
  - Certain low-level tasks are written using assembly-language instructions
- In an interactive system, it is possible to call / use two or more system calls in a command
  - Possible error conditions for each system call must be handled appropriately
- Run-Time Environment (RTE)
  - Provides the system-call interface that links API function calls to the necessary kernel-level system calls
- Passing Parameters
  - In registers (if few parameters)
  - In a block or table in memory, with the address passed in a register
  - Placed onto the stack by the program



Passing Parameters as a Table

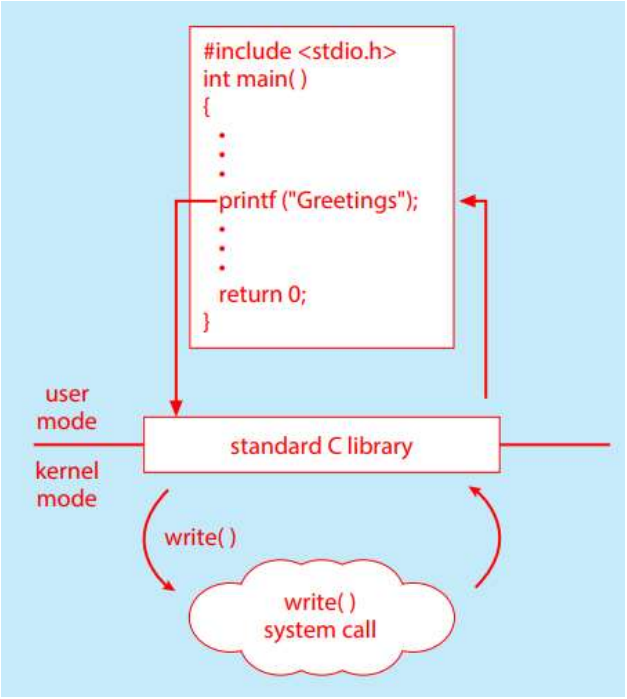


System Calls – Types of System Calls

- Process Control:
  - Manage the lifecycle of processes (creating, terminating, loading)
  - Control process attributes and memory allocation
- File Management:
  - Handle file-related operations (creating, deleting, opening, closing)
  - Enable reading from and writing to files
- Device Management:
  - Control I/O devices (requesting, releasing, reading, writing)
  - Manage device attributes
- Information Maintenance:
  - Transfer information between the user program and the OS
  - Get system data like time, date, and resource usage
- Communications:
  - Enable processes to exchange information (e.g., via message passing or shared memory)
  - Create and delete communication connections
- Protection:
  - Control access to system resources
  - Get and set file permissions

System Calls - Application Programming Interface (API)

- API's Role
  - Programmers use an API (e.g., Win32 API, POSIX API) to design their programs
  - This is because APIs are often more straightforward and abstract than the raw system calls
- Portability
  - Using a standard API allows a program to compile and run on any system that supports that same API, making the code portable across different operating systems
- API vs. System Call
  - An API function call is a high-level request
  - Behind the scenes, the run-time environment (RTE) provides a system-call interface that handles the low-level details
  - This interface intercepts the API call and invokes the necessary system call in the operating system kernel
- Example
  - A printf() function call in a C program is part of the standard C library (an API)
  - This library call is intercepted by the RTE, which then makes a low-level write() system call to the kernel to output the text to a device

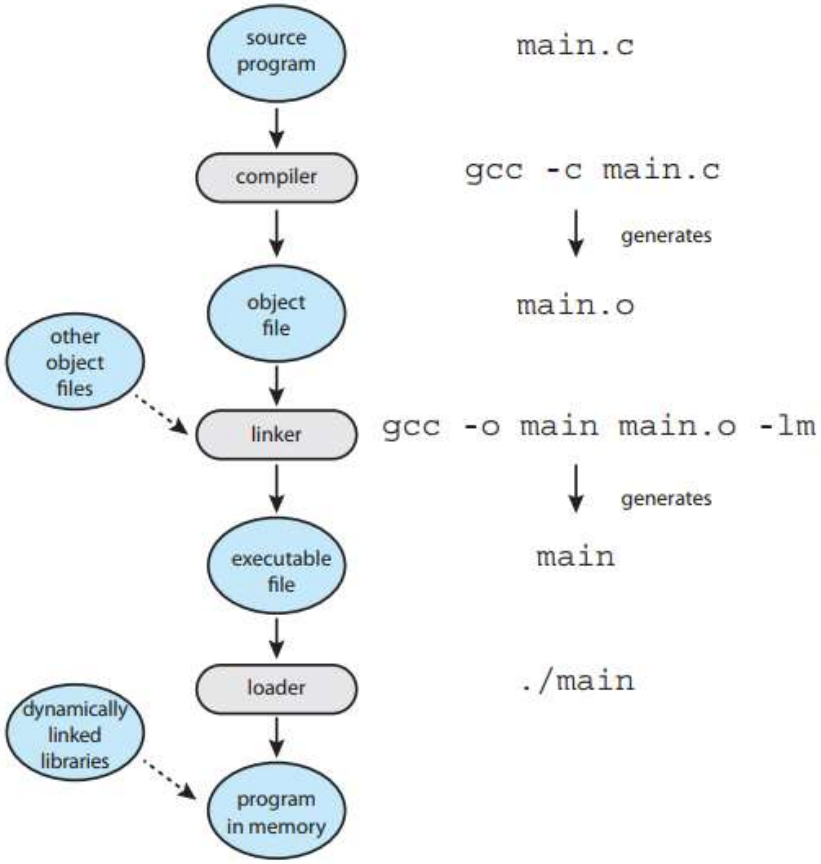


System Services

- **File Management:** Utilities to create, delete, copy, rename, and manipulate files and directories
- **Status Information:** Programs that provide system information such as date, time, memory usage, and performance statistics
- **File Modification:** Text editors and other tools for creating and modifying files
- **Programming-Language Support:** Compilers, assemblers, debuggers, and interpreters
- **Program Loading and Execution:** Utilities to load programs into memory and start their execution
- **Communications:** Programs that provide connections for sending messages, remote login, and file transfer
- **Background Services:** Long-running processes (daemons) that handle specific system tasks (e.g., network listeners, print servers)

Linkers and Loaders

- A program resides on disk as a binary executable file (a.out or prog.exe)
- To run on a CPU, the program is brought into memory and placed in the context of a process



Linkers and Loaders

- **Compiler:** Translates source code into relocatable object files
- **Linker:** Combines relocatable object files and libraries into a single binary executable file
- **Loader:** Loads the executable file into memory for execution
- **Dynamic Linking:** Libraries are not linked until execution time. This reduces the size of executable files and allows multiple processes to share a single copy of a library in memory
- **Relocation:** The process of assigning final memory addresses to program parts during linking or loading

Why Applications Are Operating-System Specific

- **Unique System Calls:** Each OS provides a different set of system calls, which are the fundamental interface for applications
- **Binary Formats:** Applications are compiled into a specific binary format (e.g., ELF for Linux, PE for Windows) that the OS loader understands
- **CPU Instruction Sets:** Applications must be compiled for a specific CPU architecture (e.g., x86, ARM)
- **APIs and ABIs:**
  - APIs provide a standardized set of functions for programmers
  - The Application Binary Interface (ABI) defines how binary code components interface for a given OS on a given architecture

Operating-System Design and Implementation

- Design Goals:
  - User Goals: Convenience, ease of use, reliability, safety, speed
  - System Goals: Ease of design, implementation, and maintenance; flexibility, reliability, efficiency
- Mechanisms and Policies:
  - Mechanisms determine how to do something (e.g., a timer for CPU protection)
  - Policies determine what will be done (e.g., how long the timer should be set)
  - Separating policy from mechanism is crucial for flexibility
- Implementation:
  - Early OSes were written in assembly language
  - Most modern OSes are written in higher-level languages (C, C++) for faster development, portability, and easier maintenance

Operating System Structure

- **Monolithic Structure:** All kernel functionality is in a single, static binary file running in a single address space (e.g., early UNIX, Linux). Efficient but difficult to manage and extend
- **Layered Approach:** The OS is broken into layers, where each layer uses the functions of only the layers below it. Simplifies debugging and maintenance but can suffer from performance overhead
- **Microkernels:** Moves nonessential components out of the kernel into user space. Communication occurs via message passing. Provides greater security and extensibility but can have performance issues
- **Modules:** The kernel has a set of core components and can dynamically link in additional services (modules) at runtime. This is a common design for modern OSes (e.g., Linux, macOS, Windows)
- **Hybrid Systems:** Combines different structures to optimize for performance, security, and usability (e.g., macOS combines a microkernel and monolithic structure; Android uses a Linux kernel with extensive middleware)

Building and Booting an Operating System

- System Generation:
  - The process of configuring and compiling the OS for a specific hardware configuration
  - Options range from full recompilation to dynamically loading precompiled modules
- System Boot Process:
  - A bootstrap program in firmware (BIOS or UEFI) is run
  - It locates and loads the OS kernel into memory
  - The kernel initializes hardware and mounts the root file system
  - The system starts daemons and presents a login prompt or graphical interface

Operating-System Debugging

- Failure Analysis:
  - Log files: Used to record system errors
  - Core dumps: A capture of a process's memory at the time of failure
  - Crash dumps: A capture of the kernel's memory at the time of a system crash
- Performance Monitoring:
  - Counters: OS keeps track of system activity through counters (e.g., number of system calls, I/O operations)
    - Tools like `ps` and `top` use these.
  - Tracing: Tools like strace and BCC collect data for specific events, such as system calls or I/O activity

Operating System Structure